

TagFex: Task-Agnostic Guided Feature Expansion for Class-Incremental Learning

Bowen Zheng, Da-Wei Zhou, Han-Jia Ye, De-Chuan Zhan

Course Instructor: Prof. Krishna Mohan,
Mentor: Rahul Biswas
Presentor: Umanshiva Ladva

CVPR, 2025

Table of Contents

- 1 Brief of Presentation-1
- 2 Replication of Results
- 3 Limitations
- 4 Novel Ideas
- 5 Conclusion and Future Work
- 6 References

Proposal

- This paper proposes to continually capture diverse features in a separate feature space, and then utilize it to help the incremental learning.
- **Task-specific Features:** In supervised classification within one task, the model only requires to capture minimal necessary features for the classification within the task. Such features are useful within a specific task, thus task-specific features.
- **Task-agnostic features:** There are plenty of features not discovered by these models containing more information , not necessarily related to the classification of the task, so task-agnostic features. If these features are available for subsequent tasks, the new task-specific model can refer to them for better feature learning.
- The proposed framework **TagFex** is composed of three parts:
 - The continual acquisition of the task-agnostic features
 - the adaptive feature aggregation with merge attention.
 - the task-agnostic guided knowledge transfer.

Overview

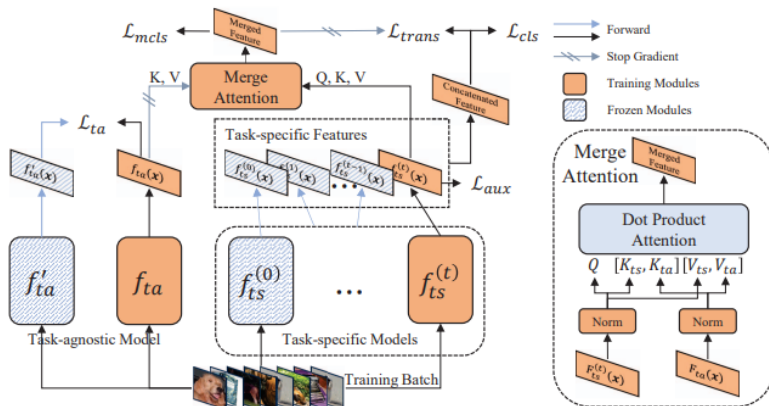


Figure: Overview of the proposed framework

Actual Results

Table 1. Performance results. Bold font represents the best results in the scenario. TagFex-P is the method with pruning strategy.

Methods	CIFAR100				ImageNet100				ImageNet1000	
	10-10		50-10		10-10		50-10		100-100	
	Last	Avg	Last	Avg	Last	Avg	Last	Avg	Last	Avg
iCaRL [34]	49.52	64.64	50.56	60.08	50.98	67.11	53.69	62.56	40.47	57.55
BiC [50]	50.79	65.38	43.82	57.04	42.40	65.13	49.90	66.36	-	-
DyTox [10]	60.43	73.50	-	-	67.61	76.51	-	-	59.75	68.14
BEEF [44]	60.98	71.94	63.51	70.71	68.78	77.62	70.98	77.27	58.67	67.09
DER [51]	64.35	75.36	65.27	72.60	66.71	77.18	71.08	77.71	58.83	66.87
TagFex	68.23	78.45	70.33	75.87	70.84	79.27	75.54	80.64	61.45	68.32
TagFex-P	67.34	78.02	69.26	74.24	69.21	78.56	74.13	79.85	60.14	67.65

Figure: Performance Results

Hyperparameters

General Configuration	My Settings	Original setting
Base Epochs	70	200
Incremental Epochs	50	170
Batch-size	128	128
Learning Rate	0.1	0.1

Table: Hyperparameter Setting

Obtained Results

Scenario	CIFAR-100 10-10	
	Last	Average
Original	68.23	78.45
Obtained Accuracy-1	66.13	74.00
Obtained Accuracy-5	88.99	93.56

Table: Replicated Results

Limitations

- Expansion modules add **extra parameters** → higher memory cost than pure regularization.
- Dominant new-class features may still bias decision boundaries.
- Requires careful balancing of **expansion vs. overfitting**.
- Theoretical analysis of long-term scalability still open.

Motivation: Why Avoid Task Expansion?

- TagFex relies on task-dependent expansions for feature extraction across tasks.
- This leads to:
 - Parameter growth with every task.
 - Increased GPU memory usage.
 - Reduced scalability for real-world CL settings.
- **Goal:** Maintain strong performance without expanding model capacity.

Learning to Prompt(L2P)

- Learning to Prompt (L2P) provides a unified mechanism for continual learning.
- No need for task-specific layers—prompts act as soft, learnable task representations.
- Key strengths:
 - Task-agnostic and scalable.
 - Efficient reuse of prompts across tasks.
 - Avoids catastrophic forgetting via prompt selection.
- **In TagFex:** replaces heavy expansion with lightweight prompts.

What L2P Does

- It learns to dynamically prompt a pre-trained model to learn tasks sequentially under different task transitions.
- Maintains a **prompt pool** of learnable vectors.
- At inference/training time:
 - Uses similarity-based retrieval to select the most relevant prompts.
 - Prepends them to input tokens (or features).
- Prompts guide the base network to adapt to new tasks without changing core weights.
- Eliminates need for per-task expansion, acting as a universal adapter.
- Due to this, it requires neither a rehearsal buffer nor test-time task identity.
- It can be generalized to various continual learning settings including the most challenging and realistic task-agnostic setting.

L2P Pipeline

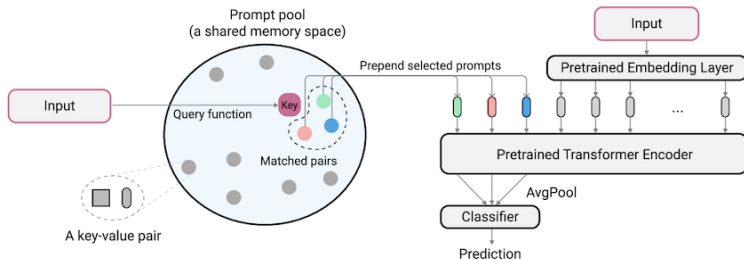


Figure: L2P

L2P Integration in TagFex

- Original TagFex uses expansion based task-specific feature extractor for each task.
- Our modification:
 - Replace the per-task feature extractor expansion with a **trainable prompt pool** while using a pre-trained ResNet18 feature extractor.
 - Extract ResNet features → concatenate relevant prompts → pass to TagFex classifier.
- Benefits:
 - Task-specific Module remains **fixed-size** across tasks.
 - Prompts specialize naturally for each task.
 - Compatible with TagFex's modular learning pipeline. Meaning that, since each part is isolated and independent module, we can swap one module with another

Memory Efficiency Advantages

- L2P removes task-specific modules which leads to no parameter explosion.
- Prompt pool is small (few thousand parameters) vs millions added per task.
- Key improvements:
 - **Lower GPU memory usage** (no task expansion).
 - **Reduced compute cost** during training.
 - **Better scalability** to 20+ tasks.

Parameter Growth

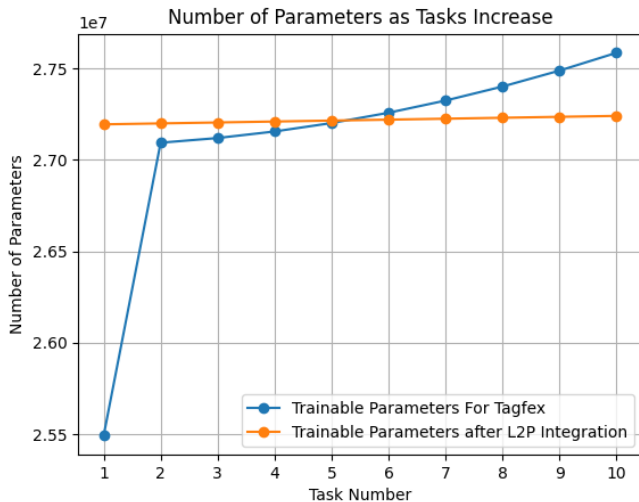


Figure: Parameter Scale before and after modification

Obtained Results

Scenario	CIFAR-100 10-10	
	Last	Average
Obtained Accuracy-1	6.47	24.26
Obtained Accuracy-5	9.72	28.85

Table: Results Obtained

- The less accuracies can be due to where we append the prompts in the pipeline.
- I appended the prompts in the task-specific module(before merging with the task-agnostic feature). This might interfere with the task-specific feature space and with the shared feature space as well when merging with task-agnostic features.
- Another reason could be TagFex relies heavily on task-dependent and shared features; replacing expansion with prompts may reduce representational capacity.

Possible Improvements

- Try to append the prompt in the merged feature space so that it does not interfere with either the task-specific and the shared feature space.
- Might want to increase the prompt pool size but could lead to less task sharing.
- Train for more epochs per task to better differentiate task-specific and task-agnostic features.
- Could use a pre-trained ViT model for better representation.
- Can shift to more robust prompt based CL approaches such as DualPrompt.

Conclusion

- Explored how **prompt-based learning** can be applied to continual learning systems to reduce memory overhead and avoid task-specific model expansion.
- The study provided insights into the challenges of integrating modular architectures with prompt-driven adaptation.
- Through experimentation and analysis, gained a deeper understanding of representation stability, feature-space interactions, and the importance of architectural compatibility.
- Overall, the project highlights both the **promise** and the **complexity** of using prompts as a scalable mechanism for continual learning as inferred from the L2P paper.

Future Work

- Explore more robust ways of combining prompts with modular CL architectures, ensuring stable shared representations.
- Investigate hybrid approaches that blend prompt tuning with lightweight adapters.
- Conduct broader experiments across diverse datasets and task configurations to study generalization and forgetting behavior.
- Analyze prompt selection dynamics and representation drift to build more reliable prompt-based CL systems.
- Extend this line of work toward **efficient, scalable, and architecture-agnostic** continual learning frameworks.

References I



Bowen Zheng, Da-Wei Zhou, Han-Jia Ye, De-Chuan Zhan “*Task-Agnostic Guided Feature Expansion for Class-Incremental Learning*”, CVPR 2025
arXiv preprint arXiv:2503.00823, 2025.



Zifeng Wang, Zizhao Zhang, Chen-Yu Lee, Han Zhang, Ruoxi Sun, Xiaoqi Ren, Guolong Su, Vincent Perot, Jennifer Dy, Tomas Pfister.
“*Learning to Prompt for Continual Learning*”, CVPR 2022
arXiv preprint arXiv:2112.08654, 2022.